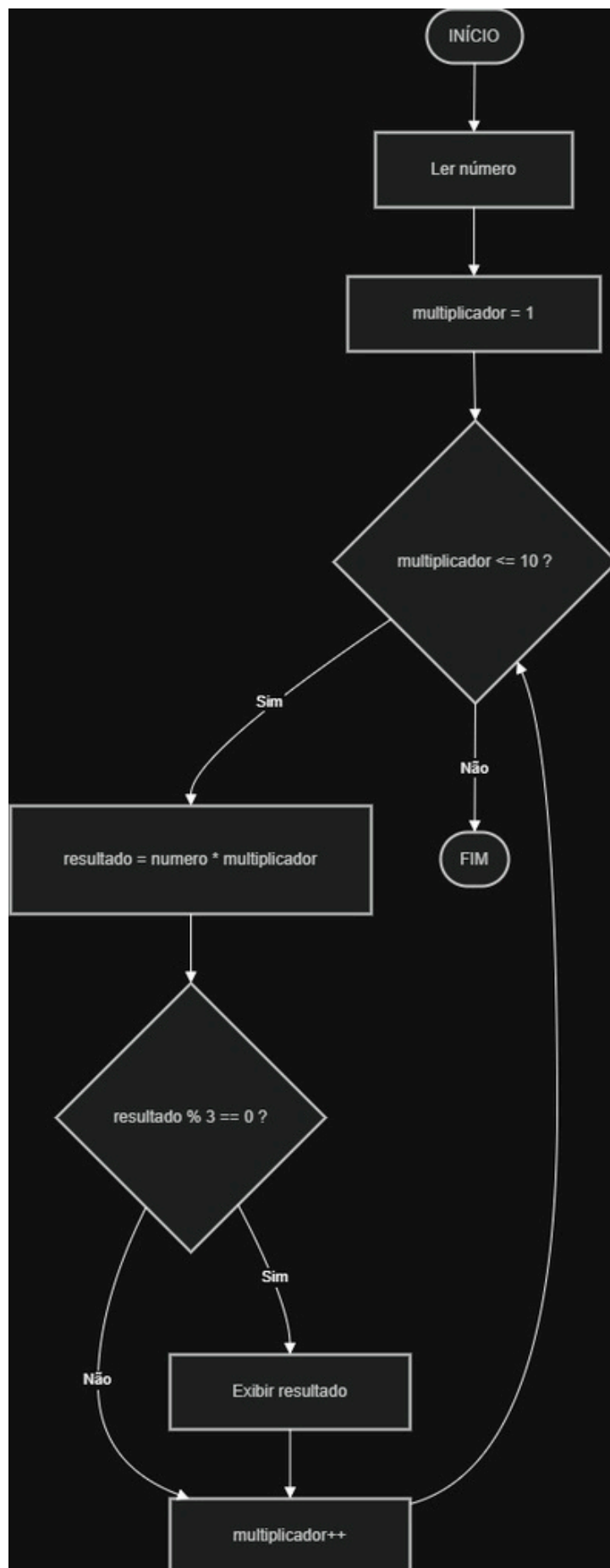


Solicite um número inteiro ao usuário e imprima a sua tabuada de 1 a 10, mas apenas os resultados que forem múltiplos de 3.

```
Multiplo.java > Multiplo > main(String[])
2
3 public class Multiplo {
4     public static void main(String[] args){
5         Scanner sc = new Scanner(System.in);
6         System.out.println(x: "Digite um número: ");
7         int numero = sc.nextInt();
8         int multiplicador = 1;
9         while( multiplicador <= 10 ){
10             int resultado = numero * multiplicador;
11             if (resultado % 3 == 0) {
12                 System.out.println(numero + " x " + multiplicador + " = " + resultado);
13             }
14             multiplicador++;
15         }
16     }
17 }
18 }
```

OUTPUT 1 DEBUG CONSOLE TERMINAL PORTS

```
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
9 x 10 = 90
PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe1> java Multiplo
Digite um número:
2
2 x 3 = 6
2 x 6 = 12
2 x 9 = 18
PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe1>
```



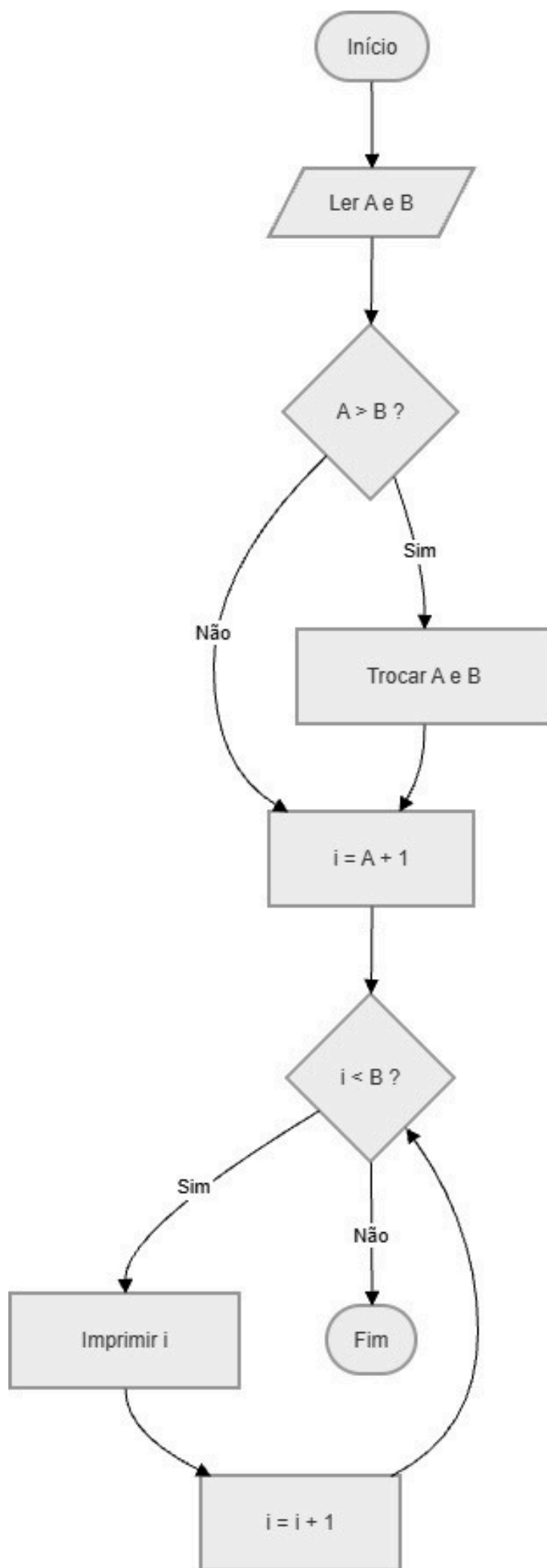
Leia dois números inteiros (a e b). Utilizando obrigatoriamente o comando while, imprima todos os números inteiros existentes entre eles em ordem crescente.

Exemplo.java > Exemplo > main(String[])

```
1 import java.util.Scanner;
2
3 public class Exemplo {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         System.out.println(x: "A e B");
7         int a = sc.nextInt();
8         int b = sc.nextInt();
9         //VERIFICA SE A É MAIOR QUE B, SE FOR, TROCA OS VALORES DE A E B
10        if (a > b) {
11            // int temp = a recebe o valor de a original
12            int temp = a;
13            // a recebe o valor de b
14            a = b;
15            // b recebe o valor de temp
16            b = temp;
17        }
18        // int i recebe o valor de a + 1
19        int i = a + 1;
20        // ENQUANTO I FOR MENOR QUE B, IMPRIME O VALOR DE I E INCREMENTA I
21        while (i < b) {
22            System.out.println(i);
23            i++;
24        }
25        sc.close();
26    }
27 }
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Ryan\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe2> java Exemplo
A e B
19 9
20
21
22
23
24
25
26
27
PS C:\Users\Ryan\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe2>

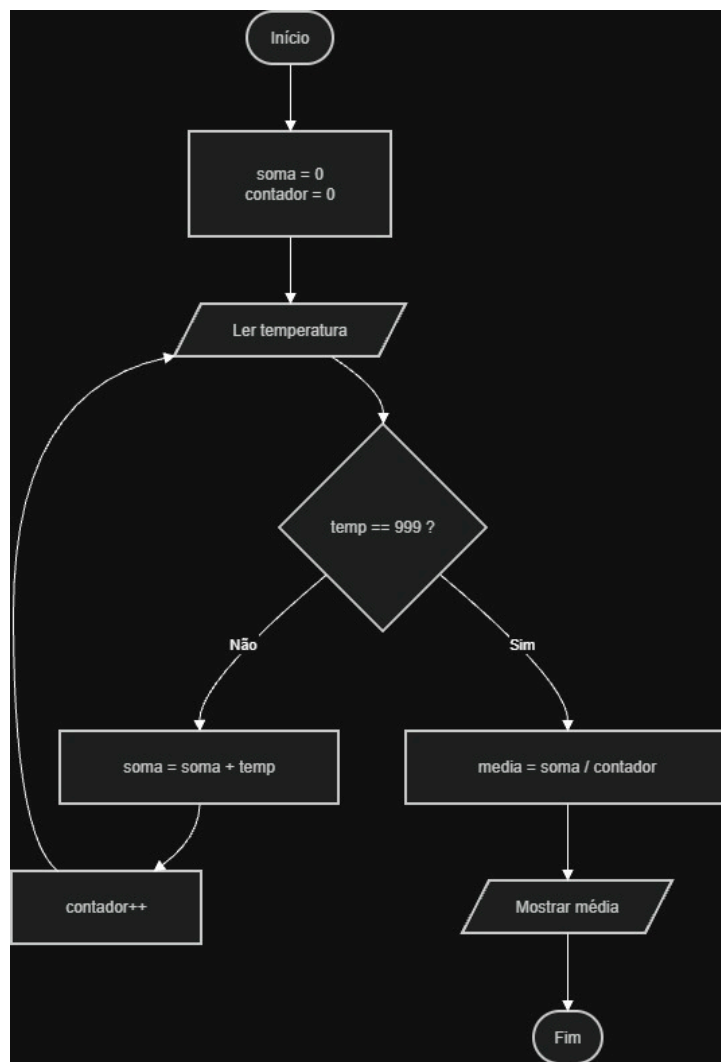


Um laboratório de solos precisa ler diversas medições de temperatura. O programa deve ler temperaturas até que o usuário digite o valor sentinela 999. Ao final, exiba a média aritmética das temperaturas válidas lidas.

```
Sentinelajava > Sentinelajava
1 //que é o sentinela? Sentinela é um valor específico que é usado para indicar o fim de um processo ou a ausência de um valor válido.
2 // No contexto de programação, uma sentinela é frequentemente usada em loops para determinar quando o loop deve parar. Por exemplo,
3 // em um programa que lê uma série de números, um valor como 999 pode ser usado como sentinela para indicar que o usuário terminou de inserir números.
4 // Quando o programa encontra esse valor, ele sabe que deve encerrar o loop e processar os dados coletados até aquele ponto.
5 import java.util.Scanner;
6
7 public class Sentinela {
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11         System.out.printf(format: "Digite a temperatura \u00B0C: ");
12         System.out.printf(format: "\n-----\n");
13         double temp1 = sc.nextInt();
14         double soma = 0;
15         double contador = 0;
16         //while para receber as temperaturas, o programa só vai parar quando o usuário digitar 999, que é a sentinela!
17         while (temp1 != 999) {
18             // todo valor que temp receber, ele vai ser somado a soma e o contador vai ser
19             // incrementado(oque vai ser usado para calcular a media (soma/contador))
20             soma += temp1;
21             contador++;
22
23             System.out.printf("Digite a temperatura novamente \u00B0C: " + "\n-----\n");
24             temp1 = sc.nextInt();
25         }
26         // proteção aqui (fora do while) para evitar divisão por zero, caso o usuário digite 999 na primeira entrada
27         if (contador > 0) {
28             double media = soma / contador;
29             System.out.println("Média: " + media);
30         } else {
31             System.out.println(x: "Nenhuma temperatura válida foi digitada.");
32         }
33         System.out.printf(format: "\n-----\n ");
34         System.out.printf(
35             "Temperatura maxima atingida \n----->\u00B0C " + "\n-----\n" +
36             "\nPrograma encerrado.",
37             temp1);
38         sc.close();
39     }
40 }
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Ryan\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe3> java Sentinela
Digite a temperatura °C:
-----
12
Digite a temperatura novamente °C:
-----
12
Digite a temperatura novamente °C:
-----
12
Digite a temperatura novamente °C:
-----
43
Digite a temperatura novamente °C:
-----
999
media lida das temperaturas citadas: 19.75
-----
```



Crie um programa que receba um número inteiro positivo e realize uma contagem regressiva até zero. A cada iteração, o programa deve exibir a mensagem:

"Sistema operando. T-menos [valor] segundos;"

```

ContagemRegressiva.java > ContagemRegressiva > main(String[])
1  import java.util.Scanner;
2
3  public class ContagemRegressiva {
4      public static void main(String[] args) {
5          Scanner scanner = new Scanner(System.in);
6          System.out.print(s: "Digite um número inteiro para a contagem regressiva: ");
7          int numero = scanner.nextInt();
8          while (numero >= 0) {
9              System.out.println("Sistema operando. T-menos " + numero + " segundo;");
10             numero--;
11         }
12         scanner.close();
13     }
14 }

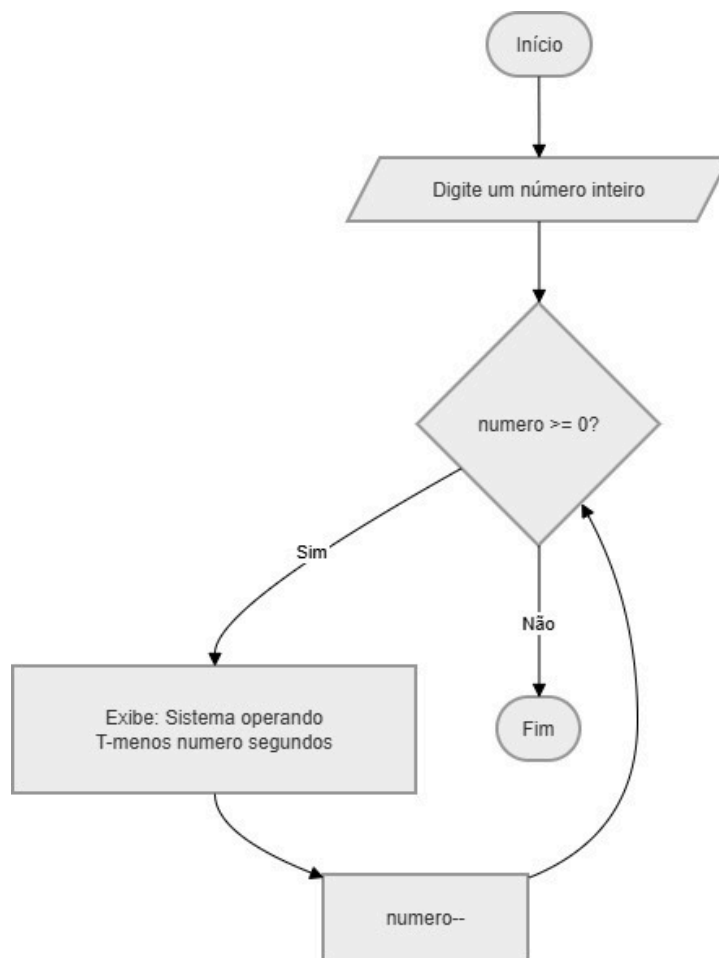
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe4> java ContagemRegressiva
Digite um número inteiro para a contagem regressiva: 12
Sistema operando. T-menos 12 segundo;
Sistema operando. T-menos 11 segundo;
Sistema operando. T-menos 10 segundo;
Sistema operando. T-menos 9 segundo;
Sistema operando. T-menos 8 segundo;
Sistema operando. T-menos 7 segundo;
Sistema operando. T-menos 6 segundo;
Sistema operando. T-menos 5 segundo;
Sistema operando. T-menos 4 segundo;
Sistema operando. T-menos 3 segundo;
Sistema operando. T-menos 2 segundo;
Sistema operando. T-menos 1 segundo;
Sistema operando. T-menos 0 segundo;
PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe4>

```



Escreva um programa que simule a validação de uma senha numérica (ex.: 2026). O programa deve permitir até 3 tentativas. Se o usuário acertar, exiba:

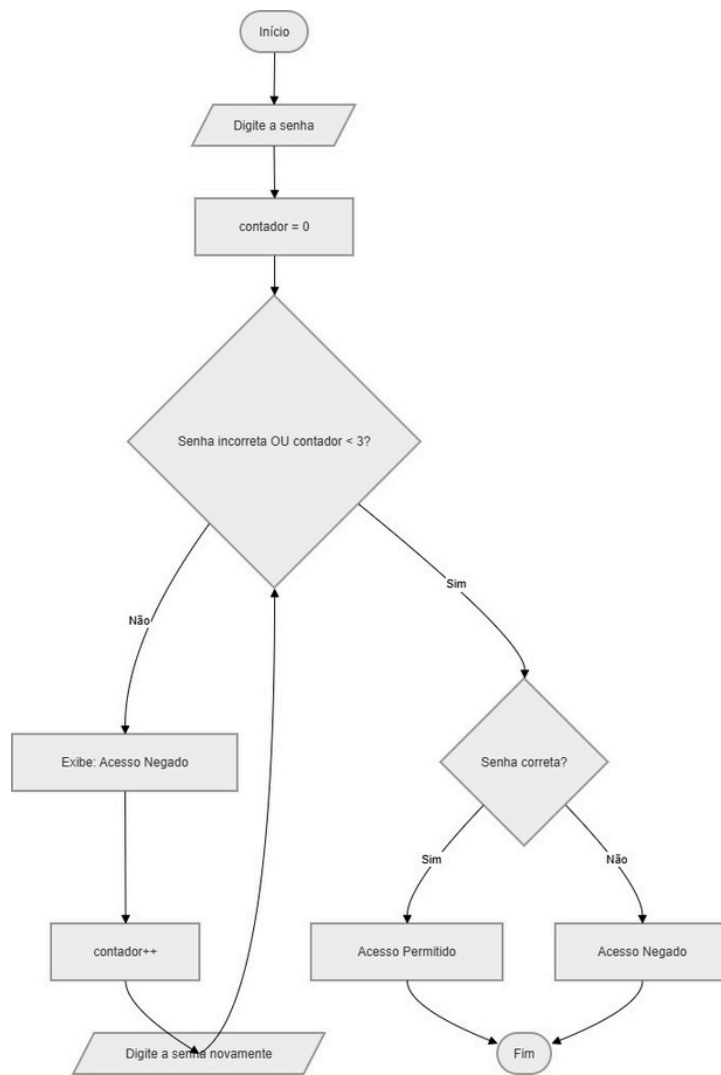
"Acesso Permitido!"

- Se esgotar as tentativas, exiba: "Terminal Bloqueado!"

```
1  import java.util.Scanner;
2
3  public class Validação {
4      Run | Debug
5      public static void main(String[] args) {
6          Scanner scanner = new Scanner(System.in);
7          String senha = "2026";
8          System.out.println(x: "Digite a senha:");
9          String ColoqueSenha = scanner.nextLine();
10         int contador = 0;
11         while (!ColoqueSenha.equals(senha) && contador < 3) {
12             System.out.println(x: "Acesso negado, tente novamente");
13             contador++;
14             ColoqueSenha = scanner.nextLine();
15         }
16         if (ColoqueSenha.equals(senha)) {
17             System.out.println(x: "Acesso permitido");
18         } else {
19             System.out.println(x: "Acesso negado");
20         }
21         scanner.close();
22     }
23 }
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe5> java Validação
Digite a senha:
1090
Acesso negado, tente novamente
2020
Acesso negado, tente novamente
2134
Acesso negado, tente novamente
1818
Acesso negado
PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe5> java Validação
Digite a senha:
2026
Acesso permitido
```

Em uma frente de pavimentação, deseja-se saber a altura média de 10 blocos de concreto. O programa deve ler as 10 alturas, uma de cada vez, e, ao final, mostrar a média das alturas e a maior e a menor altura digitada.

```

1  import java.util.Scanner;
2
3  public class Pavimentação {
4      Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7          double media;
8          double soma = 0;
9          double maior = 0;
10         double menor = 0;
11         int contador = 0;
12         while (contador < 10) {
13             System.out.print(s: "Digite a altura do bloco: ");
14             double altura = sc.nextDouble();
15             if (contador == 0) {
16                 maior = altura;
17                 menor = altura;
18             }
19             else{
20                 if (altura > maior) {
21                     maior = altura;
22                 }
23                 if (altura < menor) {
24                     menor = altura;
25                 }
26                 soma += altura;
27                 contador++;
28             }
29             media = soma / contador;
30             System.out.println("A média das alturas dos blocos é: " + media);
31             System.out.println("A maior altura é: " + maior);
32             System.out.println("A menor altura é: " + menor);
33             sc.close();
34         }
35     }

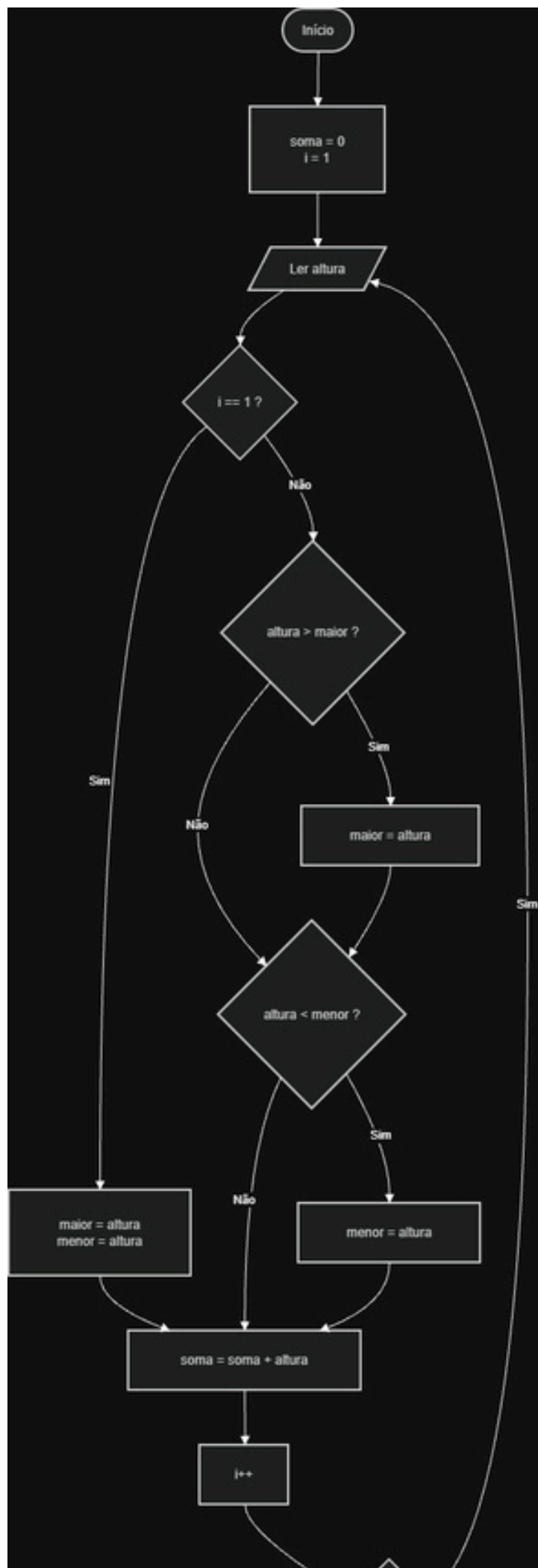
```

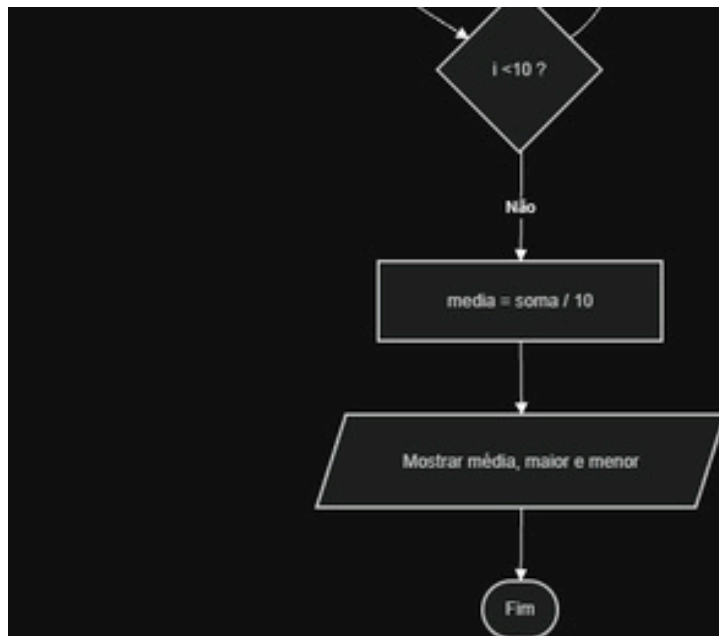
OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Digite a altura do bloco: 150
Digite a altura do bloco: 190
Digite a altura do bloco: 200
Digite a altura do bloco: 450
Digite a altura do bloco: 500
Digite a altura do bloco: 600
Digite a altura do bloco: 12
Digite a altura do bloco: 12
A média das alturas dos blocos é: 230.4
A maior altura é: 600.0
A menor altura é: 12.0

```

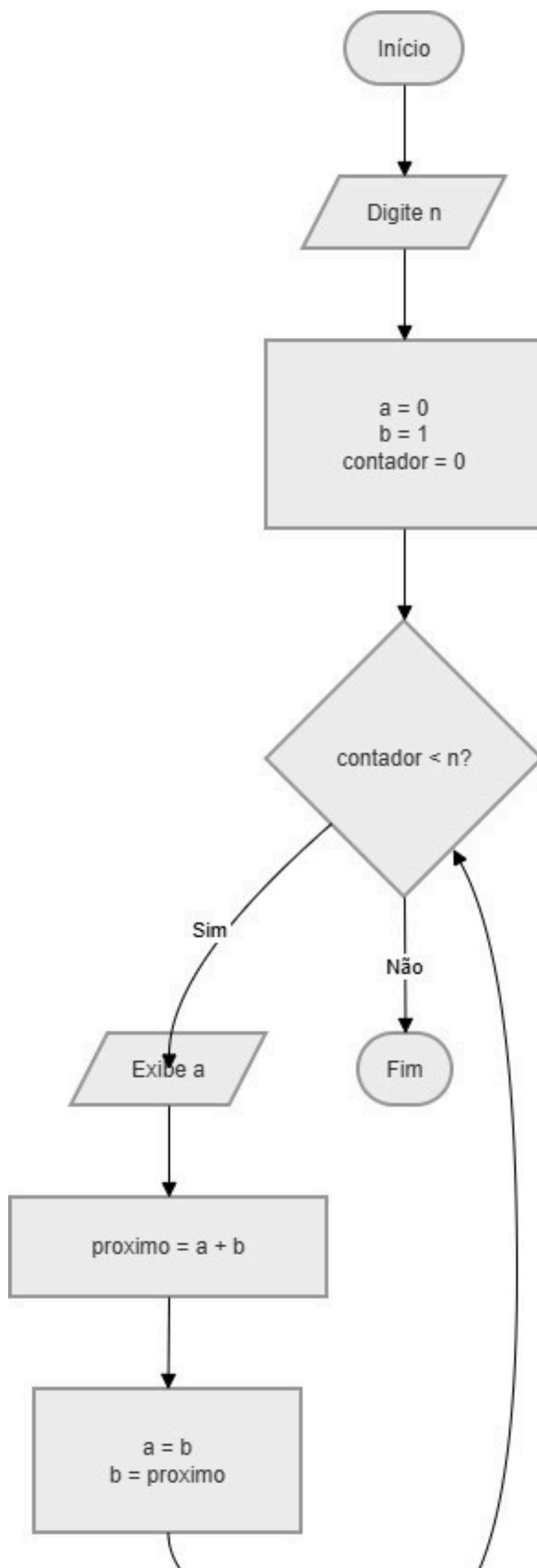




A Sequência de Fibonacci inicia com 0 e 1, e cada termo subsequente é a soma dos dois anteriores:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34 ...

- Peça ao usuário um número n e imprima os primeiros n termos da sequência usando um laço while.





```
Fubinn.java > Fubinn > main(String[])
1  import java.util.Scanner;
2
3  public class Fubinn {
4      Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7          //pedindo o valor de n para calcular a sequência de Fibonacci
8          System.out.println(x: "Digite o valor de n :");
9          int n = sc.nextInt();
10         //variáveis para calcular a sequência de Fibonacci
11         int a = 0, b = 1;
12         int contador = 0;
13         //calculando e imprimindo a sequência de Fibonacci até n
14         while (contador < n) {
15             System.out.print(a + " ");
16             // quando a sequência de Fibonacci é calculada, o próximo número é a soma dos dois anteriores
17             //entao o proximo recebe a soma de a e b, depois a recebe o valor de b e b recebe o valor de proximo, e o contador é incrementado para continuar o loop
18             int proximo = a + b;
19             a = b;
20             b = proximo;
21             contador++;
22         }
23         sc.close();
24     }
25 }
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe7> java Fubinn

Digite o valor de n : 1

0

PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe7> java Fubinn

Digite o valor de n : 10

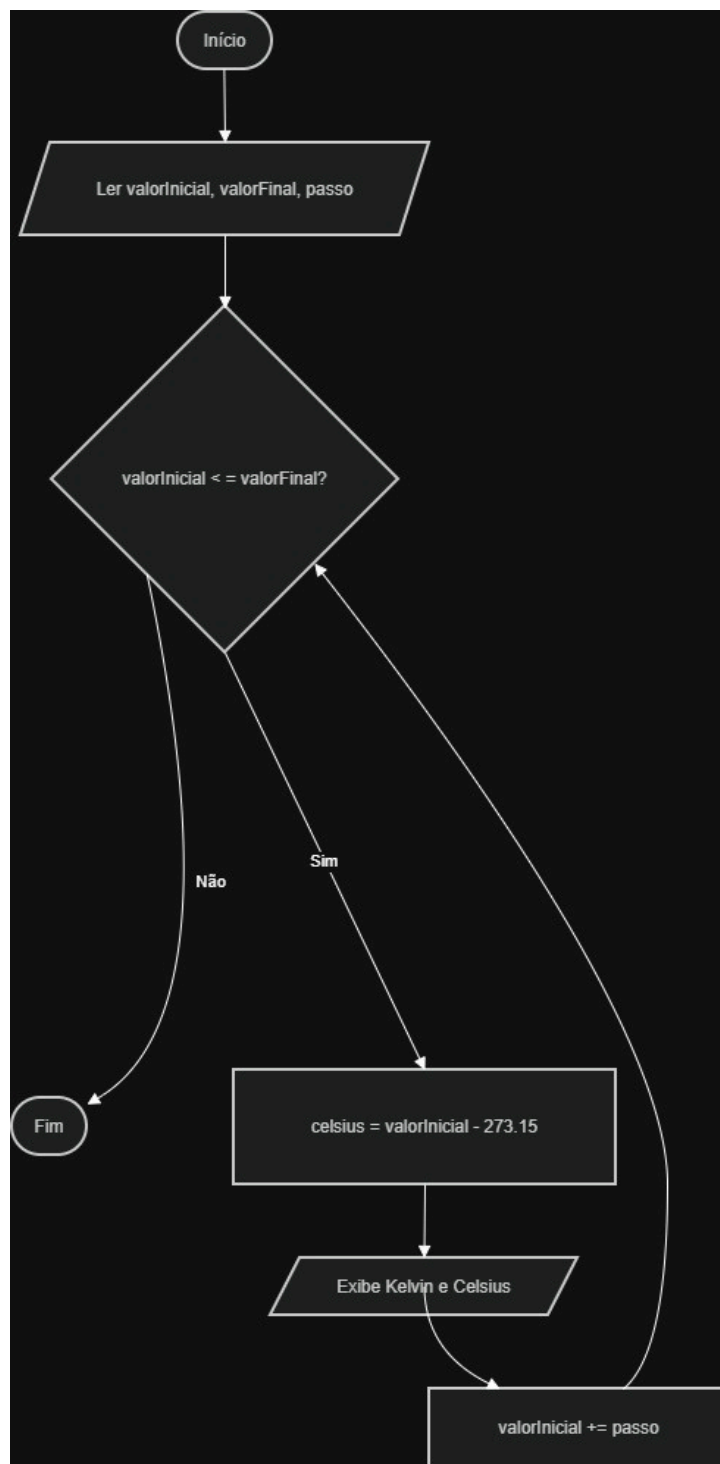
0 1 1 2 3 5 8 13 21 34

Gerar uma tabela comparativa de graus Kelvin para Celsius.

O usuário informa o valor inicial (em °K), o valor final e o passo

(incremento). O programa deve exibir a tabela formatada enquanto o

valor atual não ultrapassar o limite final.



```

4 //valor atual não ultrapassar o limite final.
5 import java.util.Scanner;
6
7 public class Kelvin {
8     Run | Debug
9     public static void main(String[] args) {
10         Scanner scanner = new Scanner(System.in);
11         double valorInicial, valorFinal, passo;
12         System.out.print(s: "Digite o valor inicial em Kelvin: ");
13         valorInicial = scanner.nextDouble();
14         System.out.print(s: "Digite o valor final em Kelvin: ");
15         valorFinal = scanner.nextDouble();
16         System.out.print(s: "Digite o passo (incremento) em Kelvin: ");
17         passo = scanner.nextDouble();
18         while (valorInicial <= valorFinal) {
19             double celsius = valorInicial - 273.15;
20             System.out.printf(format: "Kelvin: %.2f °K, | Celsius: %.2f °C%n", valorInicial, celsius);
21             valorInicial += passo;
22         }
23         scanner.close();
24     }
}

```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe8> java Kelvin

```

Digite o valor inicial em Kelvin: 80
Digite o valor final em Kelvin: 100
Digite o passo (incremento) em Kelvin: 1
Kelvin: 80,00 °K, | Celsius: -193,15 °C
Kelvin: 81,00 °K, | Celsius: -192,15 °C
Kelvin: 82,00 °K, | Celsius: -191,15 °C
Kelvin: 83,00 °K, | Celsius: -190,15 °C
Kelvin: 84,00 °K, | Celsius: -189,15 °C
Kelvin: 85,00 °K, | Celsius: -188,15 °C
Kelvin: 86,00 °K, | Celsius: -187,15 °C
Kelvin: 87,00 °K, | Celsius: -186,15 °C
Kelvin: 88,00 °K, | Celsius: -185,15 °C
Kelvin: 89,00 °K, | Celsius: -184,15 °C
Kelvin: 90,00 °K, | Celsius: -183,15 °C
Kelvin: 91,00 °K, | Celsius: -182,15 °C
Kelvin: 92,00 °K, | Celsius: -181,15 °C
Kelvin: 93,00 °K, | Celsius: -180,15 °C
Kelvin: 94,00 °K, | Celsius: -179,15 °C
Kelvin: 95,00 °K, | Celsius: -178,15 °C

```

Sem usar `Math.log()`, crie um programa que receba uma base `b` e um número `n`. Utilize um laço `while` para descobrir quantas vezes `n` pode ser dividido por `b` até chegar a um valor menor ou igual a 1 (simulando a ideia de logaritmo na base `b`).


```

1  import java.util.Scanner;
2
3  public class Base {
4      Run | Debug
5      public static void main(String[] args) {
6          Scanner scanner = new Scanner(System.in);
7          // DECLARANDO AS VARIÁVEIS
8          double n, base;
9          int i = 0;
10         System.out.print(s: "Digite um numero: ");
11         n = scanner.nextDouble();
12         System.out.print(s: "Digite a base: ");
13         base = scanner.nextDouble();
14         double valorInicial = n;
15         // -----
16         // VERIFICA SE OS VALORES SÃO VÁLIDOS
17         if (base <= 1 || n <= 0) {
18             System.out.println(x: "Valores inválidos!");
19         } else {
20             // -----
21             // CONTA QUANTAS VEZES O NUMERO PODE SER DIVIDIDO PELA BASE ATÉ CHEGAR A 1
22             while (n > 1) {
23                 System.out.printf(format: "O numero %.2f / %.2f = %.2f%n", n, base, (n / base));
24                 n = n / base;
25                 i++;
26             }
27             System.out.println("O numero " + valorInicial + " / " + base + " pode ser dividido " + i + " vezes ate chegar a 1");
28         }

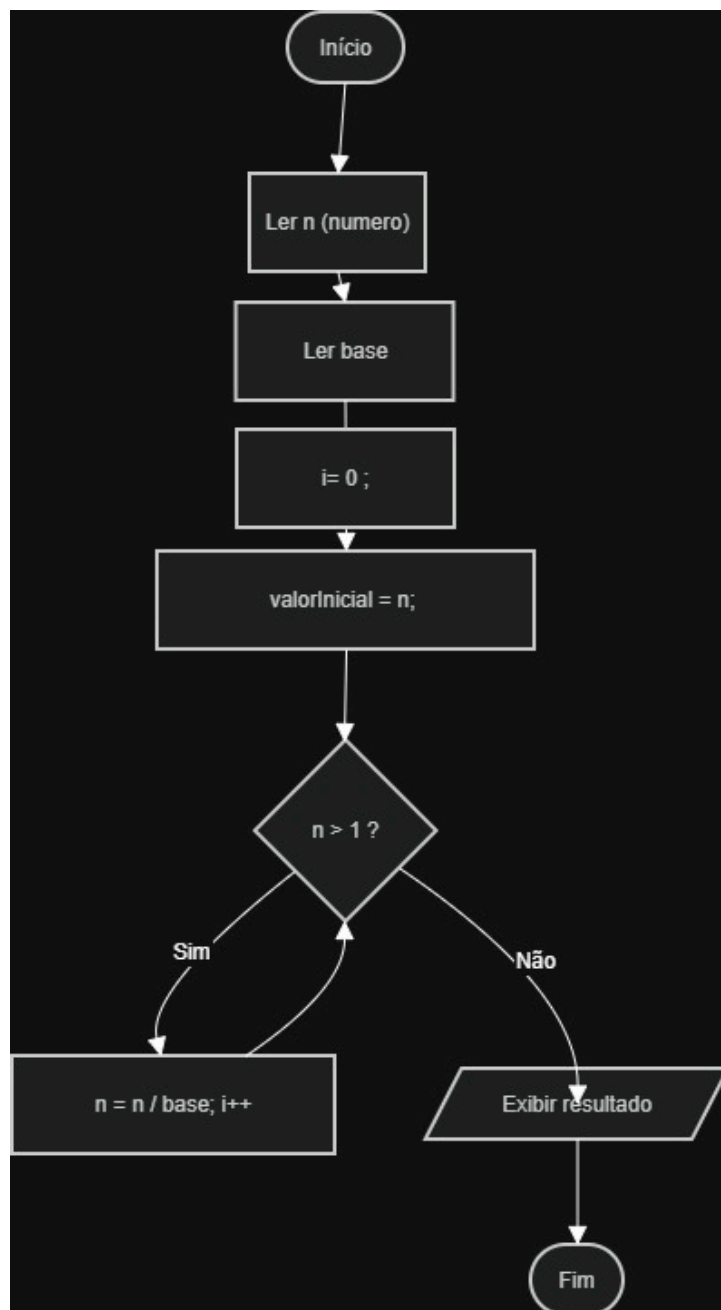
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

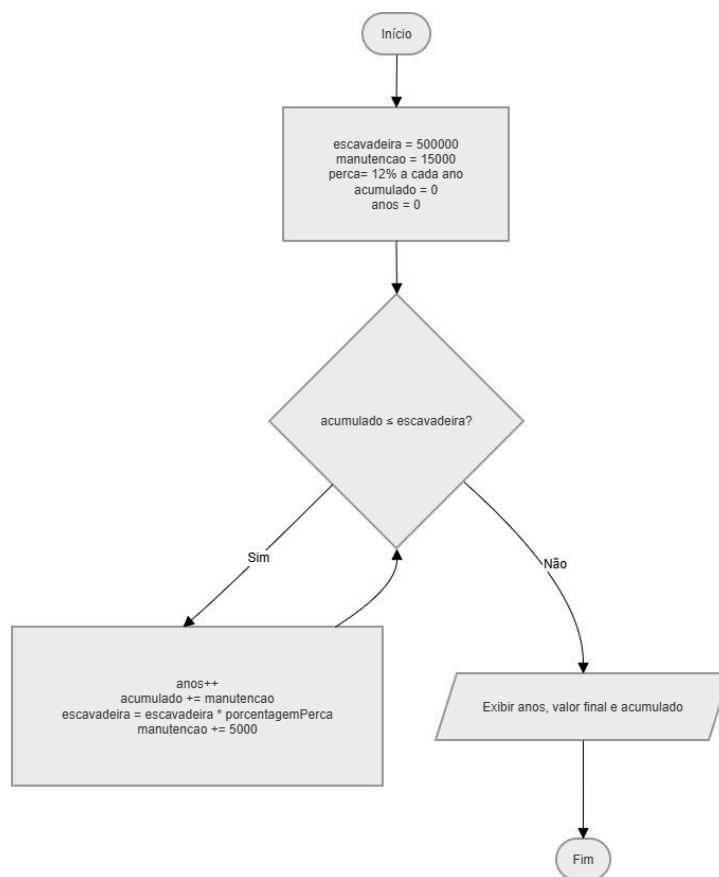
```

PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe9> java Base
Digite um numero: 10
Digite a base: 2
O numero 10,00 / 2,00 = 5,00
O numero 5,00 / 2,00 = 2,50
O numero 2,50 / 2,00 = 1,25
O numero 1,25 / 2,00 = 0,63
O numero 10.0 / 2.0 pode ser dividido 4 vezes ate chegar a 1
PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe9> 

```



Uma escavadeira custa R\$ 500.000,00. A cada ano de uso, seu valor de mercado cai 12% em relação ao ano anterior. Além disso, a manutenção anual custa R\$ 15.000,00 no primeiro ano e aumenta R\$ 5.000,00 a cada ano subsequente. Calcule em quantos anos o custo acumulado de manutenção ultrapassará o valor de mercado da máquina.



```

1 public class Trator {
  Run | Debug
2 public static void main(String[] args) {
3     double escavadeira = 500000;
4     double percentualPreço = 0.12;
5     double manutencaoAnual = 15000;
6     double acumulado = 0;
7     int anos = 0;
8     while(acumulado <= escavadeira) {
9         anos++;
10        //a cada ano, o valor da manutenção é acumulado
11        acumulado += manutencaoAnual;
12        //a cada ano, o valor da escavadeira abaixa em 12%
13        escavadeira -= escavadeira * percentualPreço;
14        //acada ano, o valor da manutenção aumenta em 5000 reais
15        manutencaoAnual += 5000;
16    }
17    System.out.println("o numero de anos pra a manutenção anual ser maior que o valor da escavadeira é: " + anos);
18    System.out.printf(format: "o valor da escavadeira é: %.2f\n", escavadeira);
19    System.out.printf(format: "o valor da manutenção : %.2f\n", acumulado);
20 }
21 }
  
```

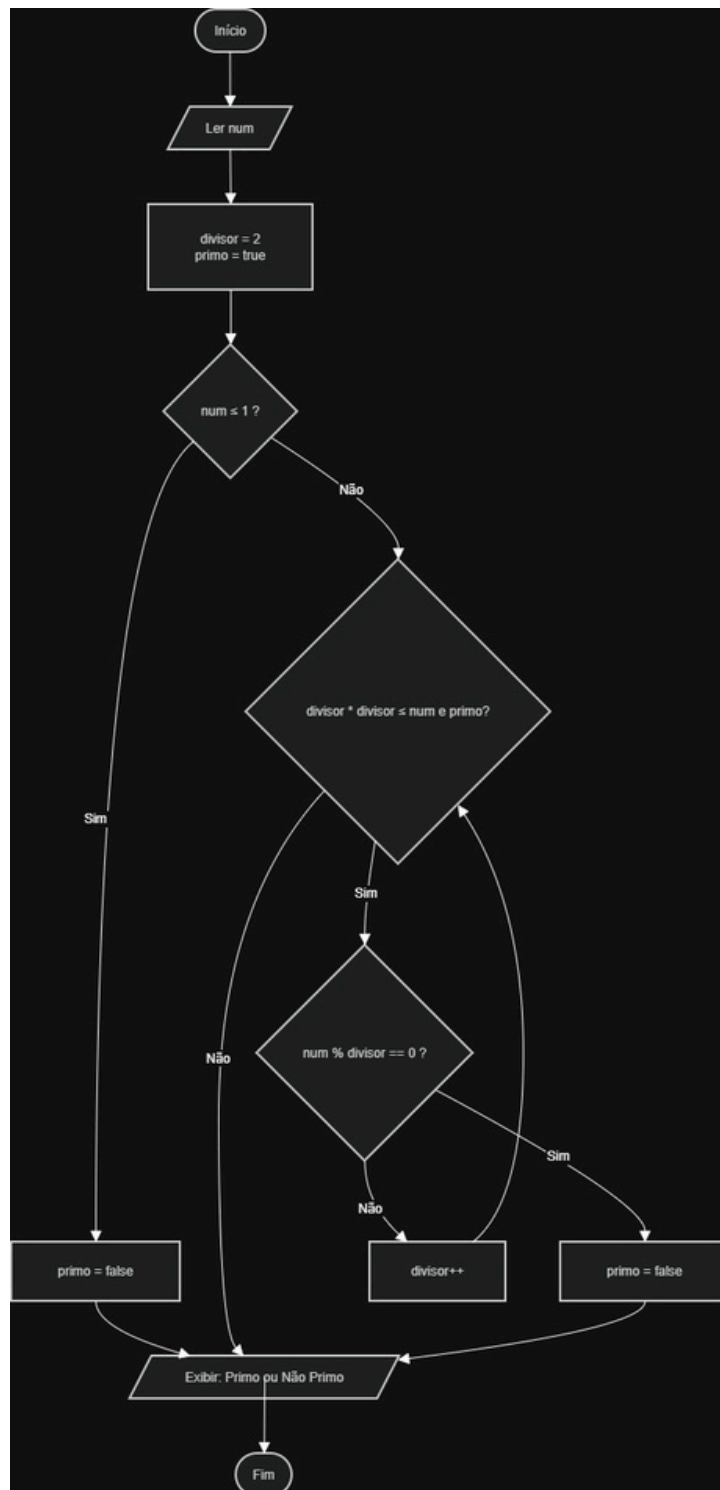
OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```

PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe10> java Trator
o numero de anos pra a manutenção anual ser maior que o valor da escavadeira é: 7
o valor da escavadeira é: 204337,80
o valor da manutenção : 210000,00
  
```

Leia um número inteiro positivo e determine se ele é um número primo. O laço while deve testar os divisores possíveis e

interromper a execução assim que encontrar um divisor ou atingir a raiz quadrada do número (otimização).



```

1  import java.util.Scanner;
2
3  public class Primo{
4      Run [Debug]
5      public static void main(String[] args){
6          Scanner sc = new Scanner(System.in);
7          int num ;
8          int divisor =2;
9          //variavel para controlar se o numero é primo ou nao
10         boolean primo = true;
11         //solicita o numero ao usuario
12         System.out.println(x: "Ingrese un numero: ");
13         num = sc.nextInt();
14         //protege contra numeros menor ou igual a 1
15         if (num <= 1){
16             primo = false;
17         }
18         //verifica se o numero é primo
19         while ( divisor*divisor< num && primo){
20             //se o numero for divisivel por algum numero entre 2 e ele mesmo, nao é primo
21             if (num % divisor == 0){
22                 primo = false;
23             }
24             //incrementa o divisor para verificar o proximo numero
25             divisor++;
26         }
27         //exibe o resultado
28         if (primo){
29             System.out.println(x: "primo");
30         } else {
31             System.out.println(x: "nao é primo");
32         }
33         sc.close();
34     }
35 }

```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Ryan\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe11> java Primo
Ingrese un numero:
nao é primo
PS C:\Users\Ryan\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe11> java Primo
Ingrese un numero:
36
nao é primo
PS C:\Users\Ryan\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe11> java Primo
Ingrese un numero:
2
primo

```

Calcular o valor por aproximação de π pela Série de Gregory-Leibniz:

$$\pi = 4 \times (1 - 1/3 + 1/5 - 1/7 + 1/9 - 1/11 + 1/13 \dots)$$

- Peça ao usuário o número de termos de precisão (1, 2, 3, 4 ...) e utilize o while para calcular o valor aproximado de π . Na tela, compare o resultado encontrado com o valor dado por Math.PI do Java. Rode o programa várias vezes, incrementando manualmente o número de termos de precisão digitado (1, 2, 3, 4 ...).

```

1  import java.util.Scanner;
2
3  public class AproximacaoPi {
4      Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7
8          System.out.print(s: "Digite o número de termos: ");
9          int termos = sc.nextInt();
10
11          int i = 0;
12          double pi = 0.0;
13          double denominador = 1.0;
14          int sinal = 1; // começa positivo
15
16          while (i < termos) {
17              pi += sinal * (1.0 / denominador);
18
19              denominador += 2; // 1, 3, 5, 7...
20              sinal *= -1;      // alterna + e -
21              i++;
22          }
23
24          pi *= 4; // multiplicação final da série
25
26          System.out.println("Pi aproximado: " + pi);
27          System.out.println("Math.PI:      " + Math.PI);
28          System.out.println("Diferença:    " + Math.abs(Math.PI - pi));
29      }
30  }

```

OUTPUT 1 DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe12> java AproximacaoPi
Digite o número de termos: 100
Pi aproximado: 3.1315929035585537
Math.PI:      3.141592653589793
Diferença:    0.00999975003123943
PS C:\Users\Ryann\OneDrive\Documentos\estudo Teorico - Calvetti\T06\exe12> java AproximacaoPi
Digite o número de termos: 10000
Pi aproximado: 3.1414926535900345
Math.PI:      3.141592653589793
Diferença:    9.99999997586265E-5

```

